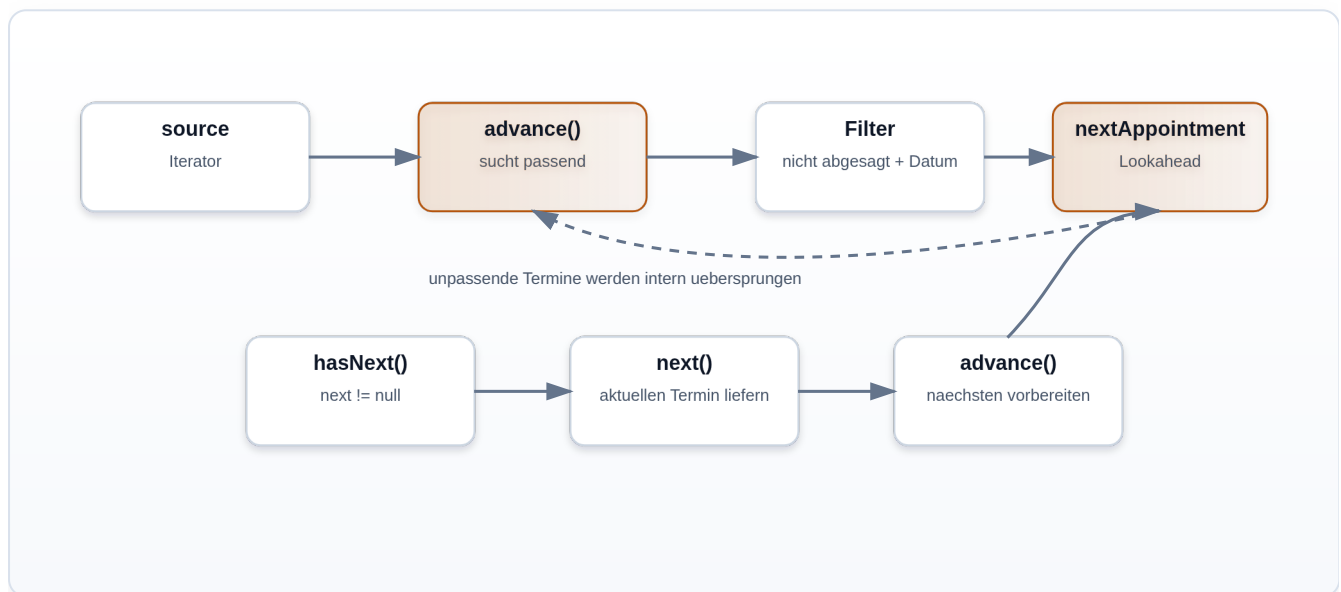


Aufgabe 04: Filternder Iterator mit Lookahead

Erklärung mit Ablaufdiagramm, zentralem Iterator-Zustand und Bezug zur konkreten Java-Implementierung.

● Professionelles Ablaufdiagramm



■ Grundidee

`TrainingCalendar` enthaelt mehrere `Appointment`-Objekte. Der Iterator soll aber nicht alle Termine liefern, sondern nur Termine, die zwei Bedingungen erfuellen:

- Der Termin ist nicht abgesagt.
- Der Termin liegt am Startdatum oder danach.

Der Client soll trotzdem ganz normal mit einem Iterator arbeiten koennen:

```

Iterator<Appointment> appointments = TrainingCalendar.sample()
    .upcomingIterator(LocalDate.of(2026, 1, 10));

while (appointments.hasNext()) {
    Appointment appointment = appointments.next();
    System.out.println(appointment.title());
}
  
```

Der Client muss also nichts ueber die Filterlogik wissen. Er fragt nur `hasNext()` und `next()` ab.

Warum reicht der normale Listen-Iterator nicht?

Der interne Listen-Iterator von `TrainingCalendar` wuerde alle Termine liefern:

- alte Termine
- abgesagte Termine
- passende Termine

Die Aufgabe verlangt aber einen Iterator, der nur passende Termine nach aussen gibt.

Dafuer kapselt `UpcomingAppointmentIterator` einen vorhandenen Iterator:

```
private final Iterator<Appointment> source;
```

`source` ist die eigentliche Datenquelle. `UpcomingAppointmentIterator` liest daraus, ueberspringt unpassende Eintraege und liefert nur passende Termine weiter.

Warum braucht die Loesung Lookahead?

Bei einem filternden Iterator muss `hasNext()` wissen, ob noch ein passendes Element existiert.

Das Problem: Das naechste Element aus `source` koennte unpassend sein. Vielleicht muessen mehrere Termine uebersprungen werden, bevor ein passender Termin gefunden wird.

Darum verwendet die Loesung ein vorbereitetes naechstes Element:

```
private Appointment nextAppointment;
```

Dieses Feld bedeutet:

- `null`: Es ist aktuell kein passender Termin vorbereitet.
- nicht `null`: Dieser Termin ist der naechste Termin, den `next()` liefern wird.

Dieses Vorauslesen nennt man Lookahead.

Die Rolle von advance()

Die Methode `advance()` sucht den naechsten passenden Termin:

```
private void advance() {
    nextAppointment = null;

    while (source.hasNext() && nextAppointment == null) {
        Appointment candidate = source.next();
        if (!candidate.cancelled() && candidate.isOnOrAfter(startDate)) {
            nextAppointment = candidate;
        }
    }
}
```

Zuerst wird `nextAppointment` auf `null` gesetzt. Danach wird die Quelle so lange gelesen, bis entweder:

- ein passender Termin gefunden wurde
- oder keine Termine mehr vorhanden sind

Unpassende Termine werden einfach nicht gespeichert. Sie werden dadurch uebersprungen.

Die Bedingung fuer passende Termine steht hier:

```
if (!candidate.cancelled() && candidate.isOnOrAfter(startDate)) {
    nextAppointment = candidate;
}
```

Ein Termin ist also passend, wenn er nicht abgesagt ist und am Startdatum oder danach liegt.

■ Warum wird `advance()` im Konstruktor aufgerufen?

Im Konstruktor steht:

```
public UpcomingAppointmentIterator(Iterator<Appointment> source, LocalDate startDate) {
    this.source = Objects.requireNonNull(source);
    this.startDate = Objects.requireNonNull(startDate);
    advance();
}
```

Der Iterator bereitet sofort den ersten passenden Termin vor.

Das ist wichtig, weil `hasNext()` danach sehr einfach sein kann:

```
return nextAppointment != null;
```

Ohne diesen ersten Aufruf von `advance()` wuesste `hasNext()` beim ersten Aufruf noch nicht, ob ein passender Termin vorhanden ist.

Die Rolle von hasNext()

`hasNext()` prüft nur, ob ein passender Termin vorbereitet wurde:

```
@Override
public boolean hasNext() {
    return nextAppointment != null;
}
```

Die eigentliche Suche passiert nicht hier, sondern in `advance()`.

Das macht die Logik leichter nachvollziehbar:

- `advance()` sucht.
- `hasNext()` fragt den vorbereiteten Zustand ab.
- `next()` liefert den vorbereiteten Termin.

Die Rolle von next()

`next()` liefert den vorbereiteten Termin:

```
@Override
public Appointment next() {
    if (!hasNext()) {
        throw new NoSuchElementException();
    }

    Appointment result = nextAppointment;
    advance();
    return result;
}
```

Zuerst wird geprüft, ob überhaupt ein Termin vorhanden ist.

Wenn nicht, wird eine `NoSuchElementException` geworfen. Das gehört zum Vertrag von `Iterator.next()`.

Wenn ein Termin vorhanden ist, passiert Folgendes:

1. Der vorbereitete Termin wird in `result` gespeichert.
2. `advance()` sucht sofort den nächsten passenden Termin.
3. Der gespeicherte Termin wird zurückgegeben.

Der Zwischenschritt mit `result` ist wichtig. Wenn zuerst `advance()` aufgerufen würde, wäre der aktuelle vorbereitete Termin verloren.

Beispielablauf

Die Beispieldaten enthalten:

```
2026-01-03 Altes Planungstreffen
2026-01-10 Iterator Workshop
2026-01-11 Abgesagte Code Review
2026-01-17 Projektpraesentation
```

Der Start ist:

```
LocalDate.of(2026, 1, 10)
```

Beim ersten `advance()` passiert:

1. `Altes Planungstreffen` wird gelesen, ist aber vor dem Startdatum.
2. `Iterator Workshop` wird gelesen und passt.
3. `nextAppointment` zeigt jetzt auf `Iterator Workshop`.

Das erste `next()` liefert `Iterator Workshop` und ruft danach wieder `advance()` auf:

1. `Abgesagte Code Review` wird gelesen, ist aber abgesagt.
2. `Projektpraesentation` wird gelesen und passt.
3. `nextAppointment` zeigt jetzt auf `Projektpraesentation`.

Das zweite `next()` liefert `Projektpraesentation` und ruft wieder `advance()` auf.

Danach ist die Quelle leer. `nextAppointment` bleibt `null`, also liefert `hasNext()` ab jetzt `false`.

Zusammenfassung

Die Loesung funktioniert, weil der Iterator den naechsten passenden Termin vorbereitet speichert.

Das Feld `nextAppointment` ist der zentrale Zustand:

- Es macht `hasNext()` einfach.
- Es verhindert, dass passende Termine verloren gehen.
- Es erlaubt dem Iterator, unpassende Termine intern zu ueberspringen.

Der Client sieht davon nichts. Er verwendet nur den normalen Iterator-Vertrag:

```
while (iterator.hasNext()) {
    Appointment appointment = iterator.next();
}
```

Genau das ist der Nutzen des Iterator Patterns: Die Traversierungs- und Filterlogik bleibt im Iterator gekapselt.